

Object Oriented Programming

Assignment III



분반	설_2, 실_1
학번	2022202109
이름	이호정

문제 1번

1

문제설명

Circular Linked List 를 만들어 각각의 명령어를 수행하도록 하게 만드는 문제이다. Linked list와의 차이점은 마지막 노드가 가리키는 것이 nullptr 이 아닌 head 라는 것이다 Initialize 명령어는 입력받은 숫자들을 순서대로 원형 연결 리스트로 만들고, transfer 명령어는 A 와 B 두 노드의 인덱스 값을 입력 받은 후, A 노드의 값을 B 노드 에 전달 한다. 노드를 하나씩 옮길 때마다 10%의 끊길 확률이 존재하도록 한다.

2

알고리즘

● Initialize 함수

먼저 command로 initialize 가 들어오면, initialize 함수가 실행된다. Initialize 함수안에서는 for 문을 사용해 각각의 값들을 받으며 노드에 연결하도록 할 것이다. 원형 연결 리스트에서는 last 노드의 설정이 필요하다. 먼저 last 노드를 nullptr 로 초기화해준다. Node를 동적할당 하여 입력받은 값을 value 값으로 가지는 newNode를 생성한다. 만약 리스트에 아무것도 없으면 newNode가 head 이자 last 노드가 되도록 설정해주고, 아니라면 last->next를 new Node로 설정해주고, last는 newNode 를 가리키도록 한다. 그리고 마지막으로 last->next를 head를 가리키도록 하면 원형연결 리스트가 만들어진다. 이 과정을 for 문으로 반복하며 하나씩 리스트에 삽입하도록 한다.

● Transfer 함수

transfer 명령어가 들어오면 뒤에 들어오는 숫자 값은 from 노드의 인덱스값과 to 노드의 인덱스 값이다. 먼저 from노드와 to노드가 입력 받은 인덱스에 해당하는 노드로 이동시켜준다. 방법을 간단히 설명하자면, Node* fromNode=head로 설정해 두었다가 for안에 formNode=from->next 이렇게 한칸 씩 이동시켜 주고, from 의 인덱스 까지 반복하도록 i의 범위를 설정해 준다. 같은 방법으로 to 노드도 이동시켜준다. 이 상태에서 from 노드의 인덱스 값이 to 노드의 인덱스 값과 같아질 때 까지 이동시켜준다. Current 노드를 만들어 from 노드를 가리키게 하고, currentIndex 를 만들어 현재의 인덱스 값을 저장한다. 그 후 current 를 next 로 보내는 것을 반복한다. 한 번 while 문이 동작할 때 마다 노드 하나씩 옮겨지는 것이다. 그리고 currentIndex도 +1 씩 해준다. 여기서 주의해야 할 것은 인덱스 값이 리스트의 사이즈max 를 넘을 때인데, 예를들어 8 to 2 를 생각해보면, 8->9-> 이렇게 인덱스 값이

증가하다가 9다음엔 0이 나와야 한다는 것이다. 이런건 $currentIndex + 1$ 만해주면 구현이 안되기 때문에 나머지 연산을 활용했다. 바로 인덱스 값을 리스트의 사이즈 만큼 나눠주는 것이다.

$currentIndex = (currentIndex + 1) \% size$ 이렇게 하면, 인덱스 값을 올바르게 증가시킬 수 있다. 어차피 나누는 값이 나눠지는 값보다 작으면 나머지는 자기자신이 나올 것이기 때문에 모든 경우에 대해 올바른 index 값이 나오게 된다. 예시로 8 to 2 의 경우를 보면 $8 \rightarrow (8+1)\%10 = 9 \rightarrow (9+1)\%10 = 0 \rightarrow$ 이렇게 올바르게 넘어가는 걸 확인 할 수 있다. 그리고 from, to 가 잘 못 입력된 경우나 from과 to 가 똑같이 입력된 경우에는 경고 메시지를 출력하고 다시 입력을 받도록 처리하였다.

● Disconnection

Transfer 과정 중에 노드 하나를 이동할 때마다 10%의 확률로 disconnect 가 되어야 한다. 이는 rand 함수로 난수를 생성한 후 10으로 나눈 나머지를 0이 될 때 disconnect 되도록 하였다. 위의 transfer 함수에서 노드 하나를 옮기는 것이 반복문 한번 돌아가는 것이기 때문에 transfer 로직 안에 연결 끊김 시뮬레이션을 넣어주면 된다.

여기서 고려해야 하는 것은 바로 9번 인덱스와 0번 인덱스 사이에서 끊어졌을 때이다. 만약 9번과 0번 사이에서 연결이 끊어지면 이전 노드를 올바르게 찾지 못하기 때문이다. 따라서 $currentIndex$ 가 0 이라면, 즉 현재 노드가 head 를 가리킬 때 이전 노드는 리스트의 마지막 노드가 되어야 한다. 이것 고려해 주지 않으면 -1 번과 0 번 사이에서 연결이 끊어졌다고 나올 것이다. 따라서 현재노드 인덱스가 0 이라면 이전 노드의 인덱스는 마지막 노드의 인덱스 값인 $size-1$ 이 되도록 설정해준다. 이렇게 하면 disconnection도 완벽하게 구현된다.

● Print 함수

Current 노드 포인터를 만들어 head 를 가리키게 한 후, 반복문 안에서 current의 value 를 출력하도록 한다. 출력을 마치면 current를 한 칸 옮기도록 설정한다. 그리고 current 가 한바퀴를 돌아 다시 head를 가리키면 반복문을 종료하도록 한다.

▽ 예시 그대로 입력

```
Command : initialize 10 20 30 40 50 60 70 80 90 100
Command : print
10 20 30 40 50 60 70 80 90 100
Command : transfer 3 to 5
Command : print
10 20 30 40 50 40 70 80 90 100
Command : transfer 8 to 2
Command : print
10 20 90 40 50 40 70 80 90 100
Command : exit
```

↳ initialize, print, transfer, exit 모두 정상적으로 동작하는 걸 확인할 수 있다.

▽ 제일 멀게 transfer

```
Command : initialize 10 20 30 40 50 60 70 80 90 100
Command : transfer 0 to 9
Command : print
10 20 30 40 50 60 70 80 90 10
Command : exit
```

↳ 운이 좋게 disconnect 되지 않고, 잘 transfer 가 된 것을 확인 할 수 있다.

▽ 9번과 0번 사이에서 disconnect 됐을 때

```
Command : initialize 10 20 30 40 50 60 70 80 90 100
Command : transfer 9 to 2
Detected a disconnection between 9 and 0
```

↳ -1 번 노드와 0번 노드가 아닌, 9번 과 0번 노드에서 연결이 끊어졌다고 제대로 나오는 것을 확인할 수 있다. 연결이 끊어졌을 경우를 생각해보면 current node 는 0번 이다. 그런데 여기서 단순히 -1 을 하게 되면 -1 노드라고 나오게 된다. 따라서 current node 가 0번일 때는 예외조건을 넣어주어서 제대로 출력 될 수 있도록 하였다.

▽ 예시와 다른 입력값

```
Command : initialize 10 90 89 55 34 88 10 40 90 09
Command : print
10 90 89 55 34 88 10 40 90 9
Command : transfer 3 to 8
Detected a disconnection between 4 and 5
```

↳ 먼저 똑같은 숫자가 입력 되도 중복되거나 하지 않고 제대로 구분하여 노드에 순서대로 입력하는 걸 확인할 수 있다.

그리고 만약 09로 입력되어도 숫자 9로 인식하여 노드에 삽입하는 것을 확인할 수 있다.

4 고찰

Linked list 에 대해서는 어느정도 알고 있다고 생각했는데 circular linked list 는 또 다르게 고려해줘야 하는 부분이 있었다. 먼저 transfer 기능을 구현할 때 from 노드와 to 노드를 딱 찍고 값을 바로 전달해 주는 것이 아니라 노드를 한칸 씩 전진해야 하는 것에서 조금 고민하였다. Transfer 3 to 5 와 같이 from 노드의 인덱스가 더 작은 경우에는 상관이 없지만 transfer 8 to 2 와 같이 from 노드의 인덱스가 to 노드의 인덱스보다 큰 경우에는 원형리스크의 last 노드의 다음 노드는 head로 가야 하기 때문에 이다. 그래서 처음 생각한 방법은 경우를 두가지로 나누는 것이었다.

(1) from index < to index 인 경우 (2) from index > to index 인 경우 이렇게 나누어 생각해 주었다. 그런데 이렇게 나누게 되면 if 문도 사용해야 하고 for 문도 사용해야 하기 때문에 코드가 길어지고 가독성이 떨어지기 때문에 보다 간결한 방법을 생각해 보았다. 그래서 쓰게된 방법이 나머지 연산자를 활용하는 것이다. 이 문제에서 노드의 개수 10개로 고정되어 있다. 따라서 current index 를 사이즈로 나누면 이 index 값이 증가해 max index 인 9를 넘더라도 current index 를 올바르게 나타낼 수 있다. 예를 들어 9번 인덱스에서 +4를 해 13번 인덱스가 되었다고 생각했을 때, 13을 10으로 나눈 나머지는 3 이다. 0번인덱스부터 시작이기 때문에 +4가 올바르게 된 것을 확인할 수 있다. 그리고 9번 인덱스에서 +1 하여 10이 되었다고 가정했을 때 10으로 나눈 나머지는 0으로 0번 인덱스로 올바르게 이동된 것을 확인할 수 있다. 그리고 인덱스의 크기가 10 보다 작은 경우엔 자기보다 큰 것으로 나뉘었을 때 나머지는 자기 자신이기 때문에 이경우에도 문제가 없는 것을 확인 할 수 있다.

① from < to 와 from > to 경우를 나눠서 구현

② index % size 를 이용

따라서 transfer 할 때 8 to 2 와 같은 경우를 고려하여 해결하는 방법은 위의 두가지가 될 수 있을 것이다.

그리고 transfer 를 할 때 일정 확률로 disconnect 하는 기능을 구현할 때에도 고려해야 할 부분이 있다.

- (1) 10% 라는 확률을 어떻게 부여할 것인가 ?
- (2) Disconnect 된 노드의 인덱스 값을 어떻게 보일 것 인가?

먼저 1번은 rand 함수를 사용했다. Rand 함수를 통해 난수를 생성하고 10으로 나눈 나머지는 0~9로 10가지가 나온다. 이 중 한 숫자가 나올 확률을 10%가 된다. 따라서 나는 이 랜덤한 수가 0 이될때 disconnect 되도록 하였다. 그리고 transfer를 하면 한번 10%의 확률에 처해지는 것이 아니라 노드를 하나씩 옮길 때마다 10%의 확률이 발생해야 하는 것이기 때문에 노드를 한칸 씩 전진시키는 반복문 안에 이 10% 확률이 한번씩 발생하도록 코딩 하였다.

2번 문제 같은 경우에는 처음에 구상할 때는 발견하지 못한 부분이다. 코드를 작성하고 여러 가지 testcase 를 넣어서 실험하다가 우연히 발견한 부분이다. 9번과 0번에서 끊어질 경우에 disconnection between -1 with 0 이라고 나오는 것을 발견 했다. 이러한 문제가 발생한 이유는 disconnect 됐을 경우에 currentIndex-1 with currentIndex 이렇게 코딩했기 때문이다. 따라서 나는 prevIndex 라는 변수를 새로 만들고, currentIndex ==0 일 경우에는 삼항연산자 조건을 넣어 처리해 주었다. 그렇게 수정된 코드가 올바르게 고쳐졌는지 확인하기 위해 강제로 9번과 0번에서 끊어본 후 위의 결과 사진과 같이 올바르게 9번과 0번 이라고 나오는 것을 확인하였다.

문제 2번

1

문제설명

이 문제는 연산자 오버로딩을 활용하여 행렬 연산을 수행하도록 하는 프로그램을 작성하는 것이 제일 포인트인 문제이다. 2차원 배열을 활용하여 먼저 Matrix class 를 만들고 class 들 끼리의 연산을 정의하여 matrix 연산 결과를 보이도록 하는 문제이다.

2

알고리즘

- Matrix class

행렬의 정의와 연산을 하게 하는 핵심 로직들이 모두 class 안에 구성되어있기에 이 문제에서 Matix class 안을 제대로 채우는 것이 중요하다. 먼저 생성자로 모든 원소를 초기화 한다. 이는 초기화 하지 않고 기본 생성자만 두어도 상관없다. 그리고 row 와 col 이라는 변수는 해당 행렬의 행과 열을 나타낸다

- setValue Method

setValue 함수는 외부에서 제공된 배열을 받아 Matrix 객체 내부의 데이터 배열인 data에 값을 설정하는 기능을 수행하도록 한다. setValue 를 통해 행렬의 초기상태를 설정하거나 값을 변경하는것이 가능하게 된다. 코드를 뜯어보자면, 이 중 for 문을 사용하여 외부에 전달된 배열의 모든 자리의 원소들을 돌게 된다. 첫 번째 for 문은 행 (i) 을 돌고, 두번째 for 문은 열을 돈다. 따라서 배열의 각각 위치마다 값을 입입력하는 것 가능하게 된다.

- Operator overloading

먼저 간단히만 개념을 설명하자면, 연산자 오버로딩이란 객체지향프로그래밍에서 사용하는 기능으로서, class 에서 새로운 동작을 하기 위해 연산자를 재정의 하는 것이다. 메서드 이름은 하나 지만 파라미터를 다르게 함으로써 메서드를 여러 개 만드는 것이 가능하게 되는것이다. 같은 이름의 연산자들이 여러 기능을 수행한다는 의미이다.

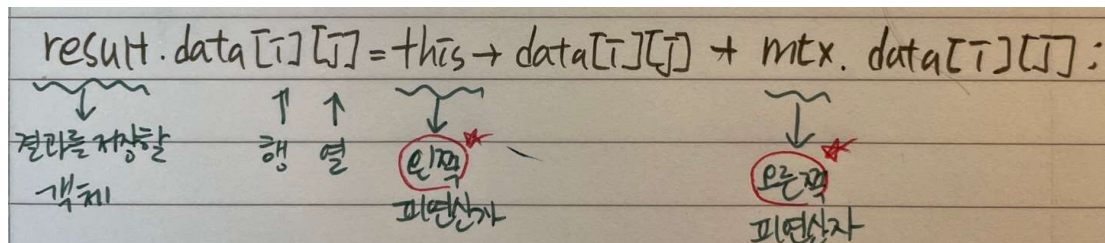
Class 끼리 더할 수 있을까? 그렇다면 class 끼리 더한다는건 무슨 의미인가? 이러한 문제에 직면하여 만들어진 것이 연산자 오버로딩이다. 이 문제와 연결지어 생각하면, matrix 1+matrix 2 를 하면 우리 인간은 그냥 행렬끼리 덧셈을 하겠지만 이

matrix 는 사실상 class 객체임으로 그냥 더하는 것이 불가능하다. 따라서 이 + 는 무엇인지 우리가 정의를 하여 행렬연산이 가능해지도록 해줄 것이다.

● Matrix & Matrix Operation

구현해야하는 행렬끼리의 연산에는 덧셈, 뺄셈 곱셈(내적)이 있다.

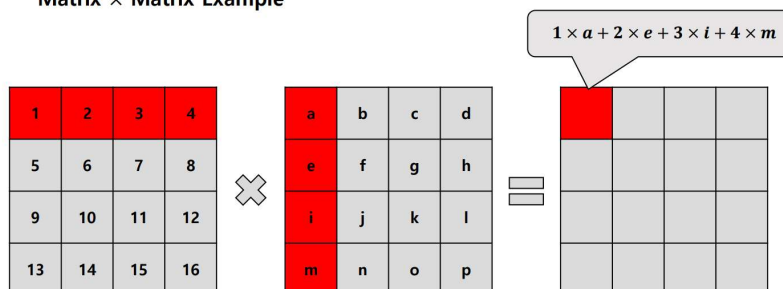
먼저 덧셈은 두 행렬의 같은 자리의 원소들끼리 덧셈을 하는것이다. 따라서 이중 for 문을 사용해 행과 열을 순회하며 각 자리의 원소를 더하고 결과에 저장하도록 한다.



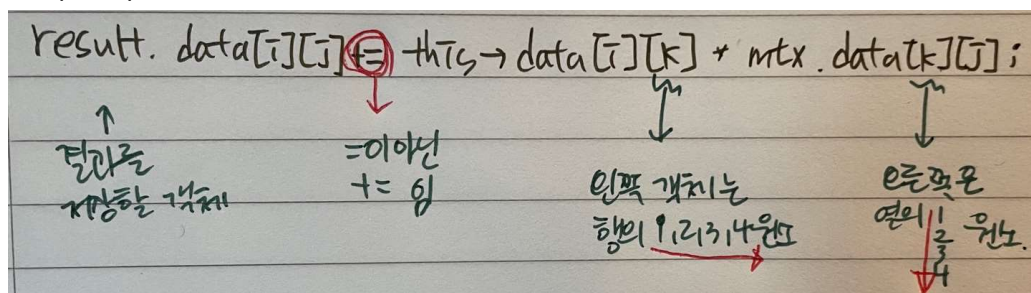
여기서 주의해야 할 부분은 현재 객체 (연산자 왼쪽에 위치한 객체)와 mtx 행렬 객체(연산자 오른쪽에 전달된 매개변수)의 위치와 역할을 주의하는것이다. 둘이 바뀌지 않도록 주의해야 한다. 물론 덧셈 연산 같은 경우엔 문제가 없겠지만 뺄셈이나 다른 연산을 할 때는 결과값이 달라질 수 있다. 그리고 연산자 오버로딩에서의 문법이 원래 연산자 왼쪽에 현재 객체를 적는 것이기 때문이다. 나중에 스칼라 연산에서도 나오겠지만 스칼라 연산 같은경우도 무조건 현재객체가 왼쪽이다.

뺄셈연산은 +를 -로만 바꿔주면 된다. 그리고 행렬의 곱셈 연산 같은경우는 행렬의 내적을 말한다. 내적은 다음과 같이 연산이 된다.

Matrix × Matrix Example



첫번째 행렬은 행, 두번째 행렬은 열의 원소들을 각각 곱하고 더한 값이 각 자리의 값이 된다.



● Matrix & Scalar Operation

행렬과 스칼라 연산은 행렬의 각 요소에 대해 스칼라, 즉 상수값을 더하거나, 빼거나, 곱하거나, 나누는 연산을 수행하는 것이다. 각 연산자에 대해 스칼라도 함께 사용할 수 있도록 연산자 오버로딩을 구현해야 한다. 그리고 역시 주의해야 할 점이 피연산자들의 위치이다. 수학적으로는 $3 * \text{matrix}$ 나 $\text{matrix} * 3$ 이나 같은 결과이고, 어떻게 연산하던 문제가 없지만 연산자 오버로딩에서는 그렇지 않다. 무조건 주체 객체인 `matrix` 를 왼쪽에 두어야 한다. 스칼라 연산 부분의 로직을 간단하게 살펴보자면 다음과 같다. 먼저 `result` 객체를 생성하고, 각 원소에 대해 스칼라 연산을 수행해준다. 이 연산을 이중 `for` 문을 이용해 모든 행과 열에 대하여 반복한다. 주의해야 할 점은 타입이 일치하는지 확인해주어야 한다.

3

결과화면

```
Matrix mat3 = mat1 + mat2;
```

```
Matrix 1:
0 0 0 0
1 1 1 1
2 2 2 2
3 3 3 3
Matrix 2:
0 1 2 3
0 1 2 3
0 1 2 3
0 1 2 3
Matrix 3:
0 1 2 3
1 2 3 4
2 3 4 5
3 4 5 6
```

▷ 문제에 나온 예시
와 같은 입력을 넣
었을 경우

`mat1+mat2` 결과가
올바르게 나오는
것을 확인할 수 있
다.

```
Matrix 1:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
Matrix 2:
16 15 14 13
12 11 10 9
8 7 6 5
4 3 2 1
Matrix 3:
17 17 17 17
17 17 17 17
17 17 17 17
17 17 17 17
```

▷ `mat1+mat2`
행렬의 덧셈 연
산을 하고 결과
를 `mat3`에 저장
해서 출력했다.
올바르게 출력되
는것을 확인할
수 있다.

```
Matrix 1:  
2 3 4 5  
6 7 8 9  
9 12 15 18  
3 9 0 2
```

```
Matrix 2:  
1 3 2 1  
0 1 1 0  
3 2 0 1  
2 1 0 3
```

```
Matrix 3:  
1 0 2 4  
6 6 7 9  
6 10 15 17  
1 8 0 -1
```

```
Matrix mat3 = mat1 - mat2;
```

▷ mat1-mat2 행렬의 덧셈 연산을 하고 결과를 mat3에 저장해서 출력했다. 올바르게 출력되는것을 확인할 수 있다.

```
Matrix mat4 = mat1 * mat2;
```

```
Matrix 1:  
2 3 4 5  
6 7 8 9  
9 12 15 18  
3 9 0 2
```

```
Matrix 2:  
1 3 2 1  
0 1 1 0  
3 2 0 1  
2 1 0 3
```

```
Matrix 4 (Matrix1 * Matrix2)  
24 22 7 21  
48 50 19 41  
90 87 30 78  
7 20 15 9
```

▷ 행렬의 곱연산, 즉 내적의 결과가 올바르게 나오는 것을 확인할 수 있다.

스칼라 연산 : 아래 Matrix 1 과 스칼라 3의 연산 결과이다

```
Matrix1 ScalarAdd with 3
5 6 7 8
9 10 11 12
12 15 18 21
6 12 3 5

Matrix1 ScalarSub with 3
-1 0 1 2
3 4 5 6
6 9 12 15
0 6 -3 -1

Matrix1 ScalarMul with 3
6 9 12 15
18 21 24 27
27 36 45 54
9 27 0 6

Matrix1 ScalarDiv with 3
0.666667 1 1.33333 1.66667
2 2.33333 2.66667 3
3 4 5 6
1 3 0 0.66667
```

```
Matrix 1:
2 3 4 5
6 7 8 9
9 12 15 18
3 9 0 2
```

2번 문제를 풀면서 처음으로 연산자 오버로딩에 대해 공부하였다. 객체 끼리의 연산이 가능할까라는 생각은 해본적이 없었지만 행렬 같은 경우에는 인간이라면 자연스럽게 연산할 수 있지만, 컴퓨터는 그렇지 않다는 것을 새삼 다시 느꼈다. 연산자 오버로딩에 대해 공부하고 문제를 푸니 재밌었다. 어렵지 않은 문제라고 생각했는데 생각보다 실수가 잦은 문제였다.

특히 연산자를 정의하며 행렬연산을 하나하나 해주고 값을 return 할때 return의 위치를 한 줄 위에 적었다. 처음에는 원인을 몰라 한참 헤맸다. 그리고 이런 어이없는 실수를 찾아내고 얻은 깨달음은 오류가 났을 때, 계산이 제대로 안될 때 이를 찾는 방법이 중요하다는 것이었다. For 문안에다가 연산되는 것들의 값을 출력을 먼저 해본다. 그러면 연산이 잘못 수행된 것인지 아니면 for 문이 예상치 못하게 중단된 것인지 알 수 있다. 나 같은 경우에는 1번만 출력되고 종료되는 것을 확인할 수 있었다. 따라서 for 문 안에서 반복이 이상하게 되었다는 것을 추측 할 수 있었다. 이렇게 코드가 왜 잘못 되었을까? 논리적으로 사고하는것도 중요하지만, 왜 틀렸는지 찾아 나가는 방법을 기르는것도 중요하다는 것을 깨달았다.

그리고 연산이 되지 않았던 이유에 대해 말하자면 return의 위치가 잘못된 것이었다. 그래서 다시금 기초로 돌아가 return 의 역할에 대해 생각해 보았다. Return 의 역할은 1. 함수를 종료한다. 2. 값을 반환한다. 이렇게 두가지이다. 따라서 첫 번째 칸 연산을 수행한 후 바로 함수를 종료하고 해당 result를 반환했기 때문에 제대로 된 값이 나오지 않았던 것이다.

문제 3번

1

문제설명

회원정보 관리 프로그램을 만들어, 로그인 / 회원가입 / 회원탈퇴 / exit 기능을 구현하는 문제이다. 멤버 정보는 struct 구조체를 만들어 저장하고 회원 정보를 관리하는 class 를 만들어 여러 기능을 수행하도록 하는 함수를 구현하는 문제이다. 각각의 기능마다 제약조건이 존재한다.

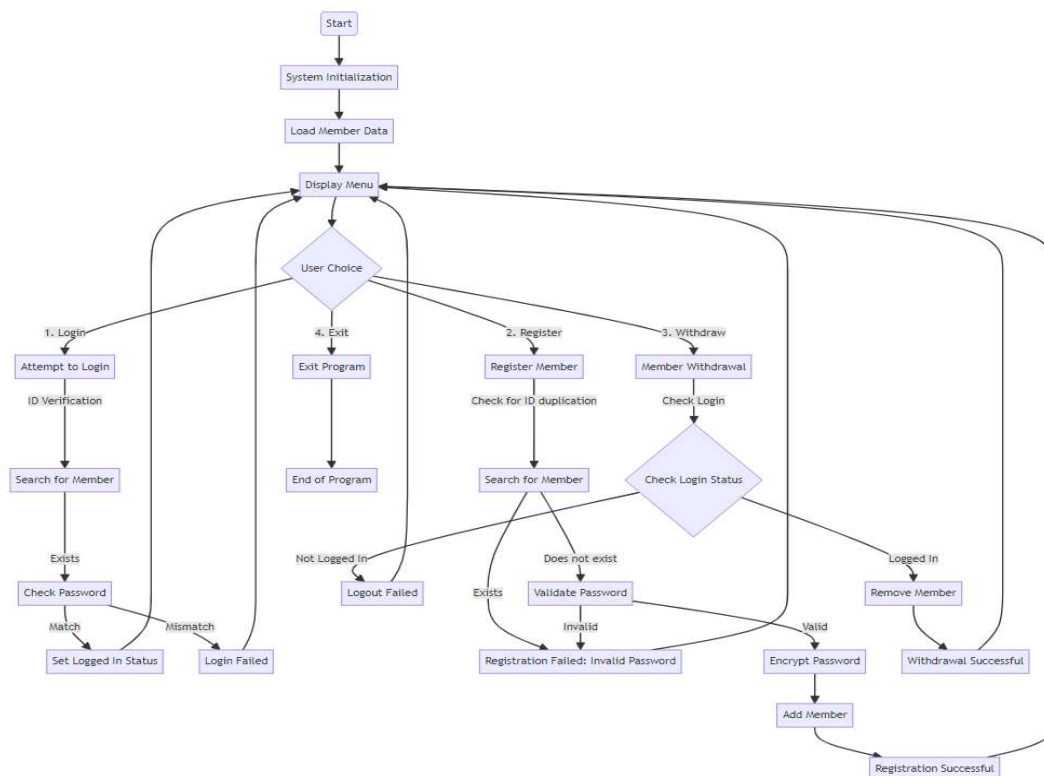
먼저 로그인 기능은 사용자의 ID 와 비밀번호가 회원정보 파일에 있는 정보와 같을 때에만 로그인이 가능하도록 해야한다. 그렇지 않은 경우에는 오류 메시지를 출력하고 명령어를 다시 입력받도록 해야 한다.

회원 가입 기능은 새로운 member 를 등록하는 기능이다. 여기서 주의해야 하는 것은 중복되는 ID 는 회원가입이 불가능 하도록 해야한다. 그리고 비밀번호에는 알파벳, 숫자, 특수기호 등 제약조건을 충족해야 한다. 그리고 비밀번호를 저장할 때는 시저 암호에 기반해 암호화 시켜 file 에 저장해야 한다.

회원탈퇴 기능은 회원 정보를 삭제하는 기능이다. 회원탈퇴는 해당 아이디로 로그인한 상태에서만 가능하다.

2

알고리즘



● Login

▷ ID exist 여부 확인

먼저 아이디와 비밀번호를 입력받는데 ID 가 이미 존재하는지 여부를 strcmp 를 이용해 검사한다. 만약 id 가 이미 존재한다면 login 함수 안의 for 문을 탈출하고 로그인 실패 메시지를 출력한다. 그리고 id 가 기존에 존재하지 않는다면 비밀번호를 암호화 한다.

▷ Encrypt password

비밀번호는 시저 암호화 방식을 따라 비밀번호를 key 크기 만큼 아스키 코드 상에서 더해준다. 지금 입력받은 비밀번호를 암호화 하는 이유는 암호화되어 저장되어있는 비밀번호와 대조해야 하기 때문이다. 물론 기존에 저장된 비밀번호를 복호화 하는 로직을 추가해도 되지만 비효율적이라고 생각이 들어 암호화 로직 만 가지고 비밀번호를 검사할것이다. Encrypt 함수를 따로 만들었는데 이 함수를 들여다보면 로직은 다음과 같다. 먼저 입력받은 id 의 길이를 key 값으로 저장한다. 그리고 for 문을 사용해 password 의 한글자 씩 접근해 글자마다 key 값을 더해 새로운 encrypted 배열에 저장한다. 이 때 주의 해야 할 점은 아스키 코드의 max 값을 넘어갈 때이다. 아스키 코드는 0 ~ 127 까지 존재한다. 따라서 key 값을 더했을 때 128 이상이 되어 버리면 문제가 생긴다. 예를 들어 알파벳 z 같은 경우는 아스키 코드상 122 번호를 가지고 있다. 만약 10글자의 아이디가 입력되었다고 가정하면 $122+10=132$ 로 아스키 코드의 범위를 넘어가 버린다. 따라서 나는 key 값을 더해주 후 128로 나눈 값의 나머지를 저장하도록 하였다. 그렇게 되면 이 문제를 해결할 수 있다. 128보다 작더라도 나머지연산이기 때문에 문제 없다. 그리고 또 유의해야 하는점은 알파벳만 암호화해야 한다는 점이다. 알파벳이 아닌 특수기호는 암호화 하지 않는다.

이런 과정을 거쳐 암호화를 마쳤다면, 암호화된 입력 비밀번호와 기존의 암호화된 비밀번호를 비교한다. 기존 비밀번호와 대조했는데 일치한다면 loggedInUser 에 현재 id 를 저장시키고 로그인 성공 메시지를 출력한다.

● Register

▷ addMember

사용자가 main에서 2번 register 를 선택한다면 addmember함수를 실행시킨다. Addmember 함수에서의 과정은 해당 id 가 이미 존재하는지 검사하고, 입력한 password 가 유효한지 검사하고, member 를 추가함으로써 용량이 over되는지 확인하고, 입력받은 비밀번호를 암호화 하기까지 이다.

▷ findMember

입력받은 id 가 이미 있는지 검사한다. 검사는 for 문을 돌며 size 까지의 모든 id 에 대하여 하였고, 같은 글자인지 판별해 주는 strcmp 함수를 활용하였다.

▷ Password validate

Password 에 대한 제약조건은 다음과 같다.

1. 글자가 10글자 이상 20글자 이하일 것.
2. 알파벳을 포함할 것
3. 숫자를 포함할 것
4. 특수기호를 포함할 것

이 모든 조건을 충족했을 때만 bool이 true를 반환하도록 하였다. 먼저 길이는 strlen을 사용하여 글자수를 검사해주었고, 제약 범위 안에 있을 때만 true 이도록 해주었다. 길이 검사를 마쳤으면 for 문을 이용해 한글자씩 검사하면서 조건을 충족한다면 hasAlpha, hasNum, hasSpecial 플래그를 true로 바꿔줄 것이다. 그리고 모든 글자 자리에 대한 검사를 마친 후에는 has Alpha && hasNum && hasSpecial 를 해준다. 그러면 세 플래그 모두 true 일 경우에만 true를 반환하게 됨으로 유효성 검사를 올바르게 마칠 수 있다. 길이에 대한 검사를 통과한 password 만이 검사를 시행함으로 길이에 대한 조건은 이미 고려된 것이다.

▷ expand 함수

만약 member를 add 함으로써 용량이 차게 된 경우는 capacity * 2 를 해줌으로써 배열 크기를 확장 시켜준다. 이 함수는 동적 할당을 사용하여 member 배열의 크기를 두 배로 늘린다. 먼저 용량을 capacity * 2 이렇게 곱해주고 새로운 배열을 할당 받아 그 배열의 크기를 2배 곱해진 capacity 크기로 정해준다.

```
Member* newMembers = new Member[capacity]
```

▷ Encrypt password

이 부분은 위의 login 에서 설명한 것과 같은 로직이다.

비밀번호는 시저 암호화 방식을 따라 비밀번호를 key 크기 만큼 아스키 코드 상에서 더해준다. 지금 입력받은 비밀번호를 암호화 하는 이유는 암호화되어 저장되어있는 비밀번호와 대조해야 하기 때문이다. 물론 기존에 저장된 비밀번호를 복호화 하는 로직을 추가해도 되지만 비효율적이라고 생각이 들어 암호화 로직 만 가지고 비밀번호를 검사할것이다. Encrypt 함수를 따로 만들었는데 이 함수를 들여다보면 로직은 다음과 같다. 먼저 입력받은 id 의 길이를 key 값으로 저장한다. 그리고 for 문을 사용해 password 의 한글자 씩 접근해 글자마다 key 값을 더해 새로운 encrypted 배열에 저장한다. 이 때 주의 해야 할 점은 아스키 코드의 max 값을 넘어갈 때이다.

아스키 코드는 0 ~ 127 까지 존재한다. 따라서 key 값을 더했을 때 128 이상이 되어 버리면 문제가 생긴다. 예를 들어 알파벳 z 같은 경우는 아스키 코드상 122 번호를 가지고 있다. 만약 10글자의 아이디가 입력되었다고 가정하면 $122+10=132$ 로 아스키 코드의 범위를 넘어가 버린다. 따라서 나는 key 값을 더해준 후 128로 나눈 값의 나머지를 저장하도록 하였다. 그렇게 되면 이 문제를 해결할 수 있다. 128보다 작더라도 나머지연산이기 때문에 문제 없다. 그리고 또 유의해야 하는점은 알파벳만 암호화해야 한다는 점이다. 알파벳이 아닌 특수기호는 암호화 하지 않는다

● Withdraw

▷ loggedInUser

회원탈퇴는 로그인된 상태에서만 가능하다. 따라서 이를 구현하기 위해 지금 로그인한 id 를 저장하는 문자열 loggedInUser를 만들었다. 그리고 만약 이 loggedInUser 아무도 없거나, 삭제하고자 하는 id 가 일치 하지 않을 경우에는 함수가 동작하지 않도록 구현했다. id 가 일치하는 것은 loggedInUser 와 입력받은 id 를 strcmp를 활용하여 검사해 주었고, if 문을 사용해 일치할 경우에만 삭제 로직이 작동되도록 하였다.

▷ withdraw logic

for문을 사용해 모든 member들을 검사하면서 일치하는 id 를 찾는다. 일치하는 member를 찾은 후, 해당 member의 인덱스 i 부터 시작하여 size-1 까지 모든 member 들을 한 칸씩 앞으로 이동한다. 즉 $members[j] = members[j + 1]$ 를 하는 것은 j+1 번째 요소의 값을 j번째에 복사하는 것이다. 그렇게 한 칸씩 모두 앞당기면 배열의 마지막 줄인 member[size - 1]은 더 이상 필요하지 않게 된다. 따라서 배열의 크기를 하나 줄이면 자연스럽게 마지막 요소는 접근할수없게 된다. 즉 사이즈가 하나 줄게된다. 배열의 사이즈가 줄었기 때문에 그 이후 컴퓨터에서 지워진 부분은 참조할 가능성이 전혀 없게 된다.

● Load Member

멤버 정보를 file 에서 불러오는 함수이다. 먼저 ifstream 안에 있는 함수를 활용하여 지정된 파일이름을 binary 모드로 연다. 그 명령어는 `ios :: binary` 이다. Binary 모드는 데이터를 txt 형태가 아닌, 메모리에 저장된 원시 형태로 읽고 쓰기 위해 사용된다. 따라서 확장자명이 만약 .txt 라고 해도 글자처럼 읽을 수는 없다.

`if(file.is_open())` 를 통해 파일 열기에 성공하면 데이터를 읽고 멤버를 추가할 것이다. 그리고 while 문을 사용하여 데이터를 읽을 것이다. `file.read((char*)&temp,`

sizeof(Member)) 을 이용하여 파일로부터 member struct 의 크기만큼 데이터를 읽을 것이다. 여기서 temp는 member 타입유형의 임시 변수이고, 파일로부터 읽은 데이터를 임시로 저장하는데 활용 할 것이다. Read 함수는 ifstram에 있는 함수로써, 읽은 만큼을 반환하고, 파일의 끝에 도달하면 더 이상 읽을 수 없게 된다. 읽고 temp에 저장하는 것을 마쳤으면 member 를 추가할것이다. Addmember 함수를 호출하여 읽은 회원 정보를 member 배열에 추가한다. 이 때, addmember의 인자로 받는 받는 것 유효성 validate와 encrypt이다. 그리고 이때 encrypt를 false 로 설정하여, 재암호화를 방지한다.

```

=====
Menu.
1. Login
2. Register
3. Withdraw
4. Exit
: 1

-----
User id: suzy1112
password: qwer12345!

-----
Login failed. Invalid User id or password
=====

```

ㄴ 로그인 실패

```

=====
Menu.
1. Login
2. Register
3. Withdraw
4. Exit
: 2

-----
User id: suzy1112
password: qwer12345!

-----
Registration successful.
=====

```

ㄴ 등록 성공

왼쪽 사진은 프로그램을 키자마자 로그인을 시도했을 때이다. 즉, 존재하지 않는 아이디로 로그인을 시도했을 때, 실패하는것을 볼 수 있다.

오른쪽 사진은 새롭게 회원 등록을 성공하는 화면이다. Password 도 유효하고, 이미 있는 ID도 아니기 때문에 성공적으로 등록 성공한 것을 볼 수 있다.

```

Menu.
1. Login
2. Register
3. Withdraw
4. Exit
: 1

-----
User id: suzy1112
password: qwer12345!

-----
Login successful.

=====
Logged in user: suzy1112
Menu.
1. Login
2. Register
3. Withdraw
4. Exit
: 3

-----
Successful withdraw.
=====

```

```

=====
Menu.
1. Login
2. Register
3. Withdraw
4. Exit
: 3

-----
Invalid.
=====

```

```

=====
Menu.
1. Login
2. Register
3. Withdraw
4. Exit
: 1

-----
User id: suzy1112
password: qwer12345!

-----
Login failed. Invalid User id or password
=====

```

왼쪽 사진은 성공적으로 회원 탈퇴에 성공한 화면이다. 탈퇴를 하기 위해서는 해당 아이디로 로그인 해야 한다.

가운데 사진은 로그인을 하지 않고 회원탈퇴를 시도했을 때 이다. 탈퇴가 불가능한 것을 확인할 수 있다.

마지막 사진은 탈퇴한 아이디로 로그인을 시도했을 때의 모습이다. 없는 아이디로 로그인을 실패하는 모습을 보면 성공적으로 탈퇴가 되었음을 알 수 있다.



왼쪽 사진은 password 조건에서 길이, 숫자, 기호 를 충족하지 않아 가입에 실패 한 화면이다.

가운데 사진은 알파벳과 숫자는 만족했지만 길이와 기호 조건을 충족하지 않았다.

오른쪽 사진은 알파벳과 숫자, 기호 조건까지 만족했지만 길이 조건인 10글자 이상이 아니므로 비밀번호 조건을 충족하지 못하였다.

3번 문제는 이번 과제에서 걸린 시간이 많기로 top3 안에 드는 문제였다. 지난 과제를 수행하면서 파일 입출력에 대해서도 어느정도 알고 있다고 생각했고, 또 class 에 대해서도 어느정도 익숙해 졌기 때문에 수월할 줄 알았지만 현실은 전혀 그렇지 않아서 많이 애를 먹었던 문제이다. 먼저 첫 번째 난관은 멤버 저장 및 읽기 방식 이었다. 멤버의 정보가 아이디와 비밀번호 두 개인데 어떤 형식으로 저장할 것이며 어떻게 불러올 것인지 고민하였다. 게다가 암호화 과정도 필요한데, 비밀번호를 암호화 하고 저장할 것인지 저장하고 암호화 할 것인지 많이 고민하였다. 그리고 두 번째는 암호화와 복호화였다. 암호화를 해본적이 없었기 때문에 일단 낫설었다. 그리고 처음에는 암호화 과정과 복호화 과정을 모두 생각해 두 함수를 모두 만들었었다. 그러나 사실 복호화 과정은 필요없다는 것을 깨달았다. 복호화는 로그인 할 때 입력한 비밀번호와 기존의 비밀번호가 일치하는지 검사하기 위해 필요한 과정이다. 그런데 굳이 이미 암호화 된 비밀번호를 복호화 해 대조하는 것이 아니라, 입력받은 비밀번호를 암호화해 이미 암호화된 비밀번호와 비교하면 되는 일이다. 따라서 복호화 과정을 없애 코드를 보다 간결하게 만들 수 있었다.

그리고 암호화 부분에서 또다른 문제가 있었다. Load member 함수에서 암호화를 고려하지 않는다면 이미 암호화된 비밀번호를 다시 암호화 하게 되고, 이러면 로그인 과정에서 비밀번호 일치 검사를 제대로 할 수 없게 된다. 따라서 이를 방지하기 위해 member 정보를 불러올때 encrypt 파라미터를 false 로 두었다. 이렇게 바뀌어 파일에서 불러온 데이터가 이미 암호화 되어 있음을 인식하고 그대로 사용할 수 있게 된다. 그리고 암호화 할 때 미처 생각하지 못한 부분이 2가지 더 있었다. 바로 알파벳만 암호화 해야 한다는 점과 아스키 코드는 127까지만 존재한다는 것이다. 처음에 그냥 password 를 key 값 만큼 더해서 저장했다. 그러나 제약 조건은 알파벳만 암호화 해야 하는 것이므로 비밀번호를 암호화 할 때 한글자씩 isalpha 인지 검사하고 맞다면 +key 를 해주고 %128 한 값을 암호화 비밀번호 배열에 저장하고 아니라면 그냥 그대로 저장하여 오류를 수정하였다. 그리고 두번째로 아스키 코드가 127까지만 있다는 것이 문제였다. 이는 위의 알고리즘 설명부분의 encrypt 함수에서 설명했듯이 아스키 코드는 0 ~ 127 까지 존재한다. 따라서 key 값을 더했을 때 128 이상이 되어버리면 문제가 생긴다. 예를 들어 알파벳 z 같은 경우는 아스키 코드상 122 번호를 가지고 있다. 만약 10글자의 아이디가 입력되었다고 가정하면 $122+10=132$ 로 아스키 코드의 범위를 넘어가 버린다. 따라서 나는 key 값을 더해준 후 128로 나눈 값의 나머지를 저장하도록 하였다. 그렇게 되면 이 문제를 해결할 수 있다. 128보다 작더라도 나머지연산이기 때문에 문제 없다. 이렇게 여러 오류를 찾아가며 코드를 작성하는 능력과 오류를 수정하는 능력 모두 향상 시킬 수 있는 기회가 되었다.

문제 4번

1

문제설명

이 문제는 지역별 시간을 관리하고 출력하는 프로그램이다. 지역별 시간은 그리니치 천문대에서 알려주는 표준 시간대를 기준으로 설정된다. 그리고 각 지역의 시간들은 time 부모 class 에서 상속받아 값을 처리한다. 그리고 요구하는 기능들을 구현해야한다. 기능들은 setting, add, print, exit 가 있다. 핵심 기능은 setting 과 add 라고 할 수 있다. Add는 초 수를 입력하면 해당 초 수 만큼 더한 값이 시간으로 setting 되도록 한다. 그리고 setting 명령어를 입력받으면 현재 시간을 동기화 하는것이다. Add 한 초수가 있다면 계산하여 setting 한다. 그리고 print 명령어를 입력받으면 모든 국가의 시간을 출력한다.

2

알고리즘

● 상속 class

▷ 모든 나라의 시간은 부모 class인 Time class 에서 시간을 상속받아 시차를 더하거나 뺀 시간이 각 나라의 시간이도록 설정한다. 이 때, 상속받기를 어떤 형태로 받았느냐에 따라 자식 class가 사용할 수 있는 class 안의 요소의 범위가 달라진다.

public 상속 (class Derived : public Base):

기본 클래스의 public 멤버는 파생 클래스에서도 public.

기본 클래스의 protected 멤버는 파생 클래스에서도 protected.

protected는 클래스 외부에서 접근 불가, 클래스내부 / 상속받아진 클래스에서 접근 가능

기본 클래스의 private 멤버는 파생 클래스에서 직접 접근 불가.

protected 상속 (class Derived : protected Base):

기본 클래스의 public 멤버는 파생 클래스에서 protected.

기본 클래스의 protected 멤버는 파생 클래스에서 protected.

기본 클래스의 private 멤버는 파생 클래스에서 직접 접근 불가.

private 상속 (class Derived : private Base):

기본 클래스의 public 멤버는 파생 클래스에서 private.

기본 클래스의 protected 멤버는 파생 클래스에서 private.

기본 클래스의 private 멤버는 파생 클래스에서 직접 접근 불가.

그걸 한눈에 표로 나타내면 아래와 같다.

Base class member access specifier	Type of Inheritance		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

● Setting

사용자가 setting 명령어를 입력하게 되면 시간을 계산하는 함수인 calculateTimes가 작동한다. 이 함수는 time_t 자료형과 tm자료형을 활용하여 동작한다.

▷ time_t 자료형과 tm 자료형

이 둘은 ctime 라이브러리에 있는, 시간과 날짜를 처리하기 위해 사용되는 데이터 타입이다.

▷ time_t 자료형

시간의 특정 지점을 표현하기 위한 자료형이다. 보통 초 단위로 표현 되며, 1970년 1월 1일 00 : 00 : 00 UTC 이후의 경과 시간을 나타낸다.

주요함수

time_t time(time_t* timer);

현재 시간을 time_t 형식으로 반환한다. 인자로 전달된 time_t 포인터가 NULL이 아니면, 현재 시간을 해당 위치에 저장한다.

time_t now = time(0);

현재 시간을 time_t 형식으로 반환한다.

time_t now; time(&now);

현재 시간을 now에 time_t형식으로 저장한다.

▷ tm자료형 (구조체)

인간이 이해할 수 있는 형태의 시간과 날짜 정보를 저장한다. 여러 개의 정수 필드를 가지며, 각각의 필드는 연도, 월, 일, 시, 분, 초 등을 나타낸다.

```
struct tm {
    int tm_sec; // 초, 범위: 0-60 (1초는 윤초를 나타낼 수 있음)
    int tm_min; // 분, 범위: 0-59
    int tm_hour; // 시, 범위: 0-23
    int tm_mday; // 일, 범위: 1-31
    int tm_mon; // 월, 범위: 0-11 (0 = 1월, 11 = 12월)
    int tm_year; // 연도, 1900년 이후의 연수 (예: 2023년이면 123)
    int tm_wday; // 요일, 범위: 0-6 (0 = 일요일)
    int tm_yday; // 연중 일수, 범위: 0-365
    int tm_isdst; // 일광 절약 시간 여부, 양수이면 일광 절약 시간 적용
};
```

▷ calculatetime

그렇게 그리니치 천문대(UTC 시간)의 시간을 받아와 각 나라의 시간으로 설정한다. 한국 같은 경우에는 UTC + 9, 워싱턴 DC의 경우에는 (UTC-4), 파리의 경우에는 (UTC+2)를 하여 setTime 함수를 각각 더하거나 빼준다. 이 때 주의해야 할 점은 더하거나 빼줄 때 24시를 넘어버리거나 -2시 이런식으로 되어버릴 수 있다는 것이다. 따라서 시차를 더하거나 빼는 연산을 마쳤을 때 24로 나누어 줌으로써 올바르게 시간을 설정할 수 있도록 한다. 한국의 경우에는 다음과 같다.

```
kor.setTime((gmt->tm_hour + 9) % 24, gmt->tm_min, gmt->tm_sec);
```

이렇게 min, sec는 그대로 받아와 설정해주지만 hour는 9시간의 시차를 계산해 주기 위해 더해 준 후 나머지 연산을 통해 항상 0~23시 사이의 값이 되게 해주었다.

```
dc.setTime((gmt->tm_hour - 4 + 24) % 24, gmt->tm_min, gmt->tm_sec);
```

워싱턴 DC 같은 경우에는 빼줘야 하는데 그러면 -N 시로 시간이 설정될 수 있기 때문에 +24를 하고 다시 24로 나머지 연산을 수행해준다.

● Add

사용자가 add 명령어를 입력하면 객체.addTime 함수를 호출된다. addTime 함수는 초수(sec)를 인자로 받는 함수이다. 입력 받은 초수를 해당 시간에 더해주고, 시간 설정을 다시 해준다.

```
this->second += sec;  
  
this->minute += second / 60;  
  
this->second %= 60;  
  
this->hour += minute / 60;  
  
this->minute %= 60;  
  
this->hour %= 24;
```

->입력 받은 초 수를 second에 더한다

->더한 초수가 60을 넘어갈 경우를 고려해 60으로 나눈 몫의 값을 minute에 더한다. 예를 들어 초가 70초가 되었을 경우 60으로 나눈 몫은 1이 된다. 이렇게 minute 설정을 완료했으면 다시 second 로 이젠 60으로 나눈 나머지 값을 second 에 저장한다. 그럼 이번엔 10초가 올바르게 설정된다.

- ➔ 그렇게 minute과 second 의 설정을 올바르게 마쳤으면 이젠 hour를 설정해준다. Hour는 minute에서 60분을 초과하여 hour로 넘어올 수 있는 값이 있는지를 확인하고 계산해 더해준다.
- ➔ 따라서 minute를 60으로 나눈 몫을 hour 에 더해준다.
- ➔ 그리고 hour에게 시간을 넘겨주고 minute을 다시 설정하기 위해 이번엔 minute을 60으로 나눈 값의 나머지를 minute에 저장한다
- ➔ 마지막으로 hour가 24시를 초과할 경우를 대비해 24로 나눈 값의 나머지를 hour로 설정하여 준다.

3

결과화면

```
Command: setting  
Command: print  
Output  
Korea = 08:59:18  
WashingtonDC = 19:59:18  
Paris = 01:59:18  
GreenwichObservatory = 23:59:18
```

▷ setting 을 입력했을 때 시간이 시간대에 맞게 설정되는 것을 확인할 수 있다.


```

Command: add 1000
Command: print
Output
Korea                = 09:15:58
WashingtonDC         = 20:15:58
Paris                 = 02:15:58
GreenwichObservatory = 00:15:58

```

▷ add 1000 했을 경우

첫 번째 사진에 1000초를 더했을 때 이다. 타이핑 하는데 시간이 좀 걸려 차이가 있겠지만 1000초는 약 16분으로 첫번째 사진의 시간에 잘 더해져서 나온다.

```

Command: print
Output
Korea                = 00:00:00
WashingtonDC         = 00:00:00
Paris                 = 00:00:00
GreenwichObservatory = 00:00:00

```

▷ setting 을 하지 않고 print 를 한 경우, 사진과 같이 초기화 (0)으로 출력되는 것을 볼 수 있다.

```

Command: add 1000000
Command: print
Output
Korea                = 23:02:38
WashingtonDC         = 10:02:38
Paris                 = 16:02:38
GreenwichObservatory = 14:02:38

```

▷ add 1000000

16666.66분, 277.77시간 이다. 하루 가 넘어가는 시간이 입력 되어도 제대로 계산이 되는 것을 확인할 수 있다.

```

Command: setting h
Command: Invalid Command
Command: add1000
Invalid Command

```

▷ 유효하지 않은 명령어

두번째 add1000 같은 경우는 띄어쓰기를 하지않아 오류가 발생

이번 문제를 풀면서 `time_t` 자료형과 상속 `class` 를 처음으로 공부하게 되었다. 그리고 `tm` 구조체를 활용하면 시간 뿐만 아니라 달력 프로그램을 만들 수 도 있겠다는 생각이 들었다.

그리고 시간 설정에 대해 많이 공부하였다. 이렇게 사용하면 그리니치 천문대의 시간을 가져올 수 있고 각각 시차를 연산하여 인자로 받아 옴으로써 여러 국가의 시간을 출력할 수 있다는 점이 흥미로웠다.

문제 5번

1

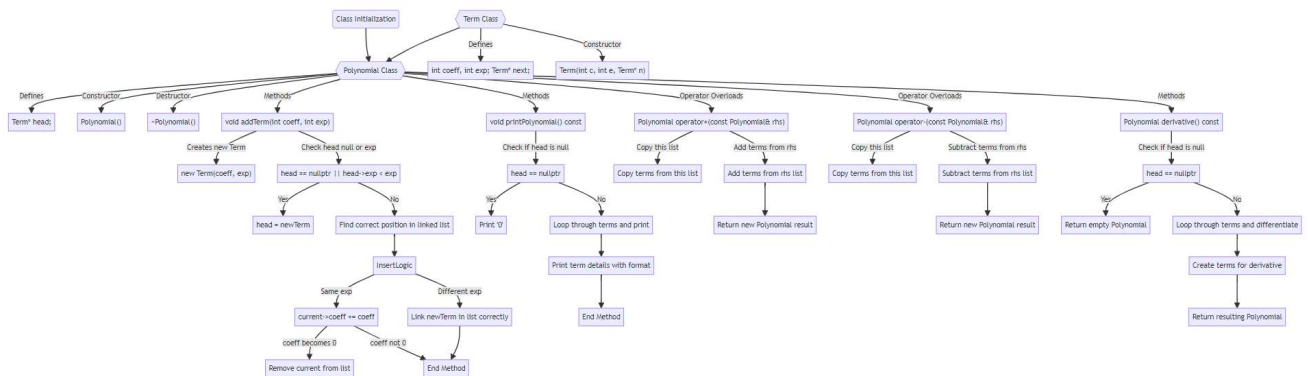
문제설명

다항식을 나타내고 연산을 할 수 있는 class 를 만드는 프로그램이다. 다항식의 각각의 항은 linked list 로 구성되어 있고, 각 항에는 계수와 차수가 존재한다. 또한 class 연산을 하기 위해 연산자 오버로딩을 구현하여 다항식끼리의 계산이 가능하도록 하는 프로그램을 구현한다. 구현해야 하는 연산에는 + 와 - 그리고 미분을 하는 기능의 함수를 구현하는 문제이다.

2

알고리즘

Flow chart _ 대략적인 흐름도는 아래와 같다.



● Term class

Term class 는 각 항을 구성하는 class 이다. Class 안에는 계수와 차수 그리고 다음 항을 연결하는 포인터가 포함되어 있다. 다항식의 모든 항을 linked list 의 node 로 관리 할 것이다.

● Polynomial class

다항식을 관리하는 class 로 다항식끼리의 연산이나 출력을 담당하는 함수들의 존재한다.

● addTerm

addTerm 함수는 다항식에 새로운 항을 추가하는 함수이다. 먼저 동적할당으로 새로운 항 newTerm 을 생성한다. newTerm 의 차수 정보를 가지고 알맞은 자리를 찾아서 넣어주는 로직을 작성한다.

```
if (head == nullptr || head->exp < exp) {  
    newTerm->next = head;  
    head = newTerm;  
}
```

먼저, 다항식이 비어있거나 newTerm 의 차수가 기존의 최고차항의 차수보다 클 때는 새로운 항이 head 가 되도록 설정한다.

대표적으로 위의 경우만 코드를 작성해 보였다. 이 외의 경우에 항의 자리를 찾아주는 로직은 다음과 같다.

먼저 current 노드를 하나 설정한다. 그리고 current 는 처음에 head 를 가리키게 한 후, 다항식이 끝날 때까지 식을 순회 하면서 새로운 항을 집어넣을 자리를 탐색한다. Current의 exp 값과 비교해 가며 current->exp 의 값이 더 크다면 계속 next 로 이동한다. 그렇게 자리를 찾았을 때 , 또 두경우로 나누어 생각해야 한다.

자리를 찾았는데 같은 차수의 항이 있는 경우, 다항식에 해당 차수를 가진 항이 없는 경우. 이렇게 두개가 있다.

먼저 첫 번째 경우에는 계수끼리 더해주면 된다. 그리고 만약 계수끼리 연산을 했는데 0이 나온다면 list 에서 항을 삭제 처리 해준다. 그렇게 newTerm 을 기존의 항에 더했을 경우에 newTerm 은 더 이상 필요하지 않음으로 삭제 해 준다.

그리고 두 번째 경우로, 자리를 찾았는데 같은 차수의 항이 없는 경우에는 새로운 항을 삽입하는 로직을 작성한다. newTerm->next = current 로 만들어주고 prev->next 는 newTerm 으로 삽입해 주면 된다. 만약 prev 가 없는 경우에는 head 를 newTerm 으로 선언한다. 이 과정을 거치면 항이 다항식에 올바르게 삽입 되는 것을 확인 할 수 있다.

● PrintPolynomial

Print의 핵심 로직은 계수인 current->coeff를 출력하고 x^{exp} 를 출력한 후 지수인 current->exp를 출력하는 것이다. 그리고 current 를 current->next 로 한 칸 옮겨주면서 반복문을 통해 다항식을 전부 출력하는 것이다. 그러나 이때 유의해야 할 점은 두 가지 있다.

먼저 다항식이 비어있는 경우 이 때는 0을 출력하도록 했다. 만약 head 가 nullptr 인경우를 if 문 조건으로 적어 처리해 주었다.

두 번째로는 계수가 음수일 때 이다. 계수가 음수 일 때 표현 할 수 있는 방법은 두 가지가 있다. -7과 +(-7) 이렇게 표현하는 방식이 있다. 문제에서는 후자의 방식으로 출력하도록 하였으니 그걸 유의하여 음수여도 모두 + 가 출력되도록 하였다.

세 번째로는 계수가 1이나 -1 인 경우에 1은 출력하지 않고 -1은 -만 출력하도록 하는 것이다.

● Operator 오버로드 함수

먼저, 덧셈 연산을 보자면 연산을 하는 다항식 term1 을 왼쪽에 두고 계산한다. 두 다항식의 덧셈을 하기 위해서는 먼저 result 객체를 생성하고 term1의 모든 항을 result 에 저장한다. 그리고 result 에 term2 의 모든 항을 저장한다. 그러면 result 에는 term1 과 term2 가 더한 결과 값이 저장된다.

뺄셈 역시 같은 로직으로 하지만 , term2 에 모두 마이너스 (-)를 곱한 후, result 에 저장한다.

마지막으로 미분 같은 경우에는 모든 항을 순회하여 검사를 진행하고, 항시 상수항 이 아닌 경우에는 미분 연산을 수행하고 결과를 result 에 저장한다. 미분 연산 하는 방식은 다음과 같다.

```
current->coeff * current->exp, current->exp - 1
```

```

Test case 1 : 기본적인 다항식 연산
Polynomial 1:  $3x^4 + 2x^2$ 
Polynomial 2:  $5x^3 + x^2$ 
Sum of Polynomials:  $3x^4 + 5x^3 + 3x^2$ 
Minus of Polynomials:  $3x^4 + -5x^3 + x^2$ 
Derivative of Polynomial 1:  $12x^3 + 4x^1$ 
Derivative of Polynomial 2:  $15x^2 + 2x^1$ 

Test case 2: 계수가 음수인 항이 있을 때
Polynomial 3:  $-4x^5 + 2x^3 + -3x^2 + x^1 + 2$ 
Polynomial 4:  $x^6 + -3x^3 + -2x^2 + 6x^1 + 4$ 
Sum of Polynomials:  $x^6 + -4x^5 + -x^3 + -5x^2 + 7x^1 + 6$ 
Minus of Polynomials:  $-x^6 + -4x^5 + 5x^3 + -x^2 + -5x^1 + -2$ 
Derivative of Polynomial 3:  $-20x^4 + 6x^2 + -6x^1 + 1$ 
Derivative of Polynomial 4:  $6x^5 + -9x^2 + -4x^1 + 6$ 

```

▷ testcase 1 : 기본적인 다항식 연산

먼저 위의 poly1, poly2 처럼 간단한 형태의 다항식 두개를 생성한 후 각각 두 다항식의 합과 차 그리고 곱셈이 올바르게 나오는 것을 확인 할 수 있다.

그리고 각 다항식의 미분 결과도 올바르게 나오는 것을 확인 할 수 있다.

그리고 음수의 경우에도 +를 출력해야 하고, 계수가 1인 경우에는 1을 출력하지 않고 -1인 경우에는 -만 출력하고, 상수항 같은 경우에는 x^0 이 출력되지 않게 하는 제약사항 또한 잘 지켜진 것을 확인 할 수 있다.

▷ testcase 2 : 계수가 음수인 항이 있을 때

연산한 결과가 음수가 아니라 처음 입력할 때부터 계수 값이 음수가 있을 경우에도 출력과 연산이 잘 되는지 확인 해 보았다. 올바르게 출력되는 것을 확인 할 수 있다.

그리고 음수의 경우에도 +를 출력해야 하고, 계수가 1인 경우에는 1을 출력하지 않고 -1인 경우에는 -만 출력하고, 상수항 같은 경우에는 x^0 이 출력되지 않게 하는 제약사항 또한 잘 지켜진 것을 확인 할 수 있다.

```

Test case 3: 모든 항의 계수가 0이 되는 경우
Polynomial 5: 0
Polynomial 6: 0x^1
Sum of Polynomials: 0x^1
Minus of Polynomials: 0x^1
Derivative of Polynomial 5: 0
Derivative of Polynomial 6: 0

Test case 4: 높은 차수의 항 추가
Polynomial 7: x^100 + -x^99 + 3x^50 + 2x^25 + 5
Polynomial 8: 5x^100 + 3x^98 + -2x^50 + 4x^25 + x^1
Sum of Polynomials: 6x^100 + -x^99 + 3x^98 + x^50 + 6x^25 + x^1 + 5
Minus of Polynomials: -4x^100 + -x^99 + -3x^98 + 5x^50 + -2x^25 + -x^1 + 5
Derivative of Polynomial 7: 100x^99 + -99x^98 + 150x^49 + 50x^24
Derivative of Polynomial 8: 500x^99 + 294x^97 + -100x^49 + 100x^24 + 1

```

▷ testcase 3: 모든 항의 계수가 0이 되는 경우

입력 할 때부터 같은 차수의 항을 여러 개 입력하고 각 항의 계수끼리 연산을 하면 0이 올바르게 나오는지 확인 해 보았다. 올바르게 나오는 것을 확인 할 수 있다. 이로써 다항식을 연산하기 전에 항을 입력받아 다항식을 구성하는 과정 또한 잘 되었다는 것을 확인 할 수 있다.

▷ testcase 3: 높은 차수의 항 추가

차수가 매우 큰 항이 추가 되어도 모든 연산 및 출력이 올바르게 되는지 확인 하였다. 미분과 덧셈 뺄셈 모두 잘 되는 것을 확인 할 수 있다.

그리고 음수의 경우에도 +를 출력해야 하고, 계수가 1인 경우에는 1을 출력하지 않고 -1인 경우에는 -만 출력하고, 상수항 같은 경우에는 x^0 이 출력되지 않게 하는 제약사항 또한 잘 지켜진 것을 확인 할 수 있다.

이번 문제에서는 연산자 오버로딩과 class 그리고 linked list를 이용해 문제를 풀었다. 그동안 배웠던 개념들을 활용해 문제를 풀었는데 각각의 개념들을 다시 되짚을 수 있어서 좋았다. 특히 연산자 오버로딩 같은 경우에는 2번에서 문제를 풀면서 어느정도 이해를 잘 했다고 생각 했는데, 이번에 문제를 풀면서 계속해서 문법적인 부분에서 막히는 경우가 많았다. 그 경험을 하며 역시 코딩은 개념만 이해하면 되는 것이 아니라 계속해서 활용을 해야 내 것으로 습득할 수 있는 것이라는 걸 깨달았다. 그리고 특히 이번 문제에선 디테일 한 부분에 많은 애를 먹었다. 예를들자면 다항식을 출력하는데 0승일 경우에는 x의 제곱 파트를 출력하지 않고, 계수가 1일 때도 1을 출력하지 않고 -1 일 때에는 - 만 출력하도록 하는 것이 은근 까다로웠다. 그러나 문제를 해결해나가면서 이런 디테일 한 부분들의 제약사항들을 충족해 나가는 힘을 기를 수 있었던 것 같아 좋았다.

문제 6번

1

문제설명

큰 틀은 “무궁화 꽃이 피었습니다.(Green Light, Red Light!)” 게임을 터미널 창에 구현하는 것이다. 게임에는 player 와 tagger 가 존재한다. Player 는 사용자로, tagger(술래) 에게 잡히지 않고 결승선 까지 전진을 모두 완료하면 이기는 게임이다. Back class 는 tagger 가 뒤 돌아보는 것을 나타내고, front class 는 tagger 가 앞을 보는 것을 나타낸다.

규칙은 아래와 같다.

- Back : 술래가 등을 돌렸을 때 green light 을 출력하고 플레이어가 움직일 수 있는 상태가 된다. 이 때 1초마다 green light 메시지를 출력하고 2초~6초 동안 랜덤하게 back 상태가 지속된다. 메시지가 느낌표를 출력하는 즉시 술래는 앞을 보게 되고 front 상태가 된다.
- Front : 술래가 앞을 볼 때 Red Light 을 출력하고 플레이어는 움직일 수 없다. 움직이게 되면 game over 이다. 이 때는 2 ~ 10 초가 랜덤하게 지속된다.
- 방향키를 눌러 앞으로 전진하는데 20번을 전진하면 승리한다. 화면에는 플레이어의 진행 상황을 보여주며, 움직임마다 @@ 로 표시하여 게임 진행도를 나타낸다.

2

알고리즘

이 코드에서 구현해야 하는 핵심 로직 세가지는 다음과 같다. 1초에 한번씩 처리하는 방법, toward 를 1초에 한번 씩 입력받는데 투워드가 어떤 투워드일지 고려해야 한다. 이것은 class 의 다형성과 관련이 있다. 그리고 redlight greenlight 언제 변형되는, turn 될 때 로직이 중요하다.

먼저 TaggerState class (상태 추상)에 대해서 설명하자면 순수 가상 함수를 정의를 사용하는데, 이는 상태의 패턴을 이용하여 술래의 상태를 관리 할 수 있다. 예를들어 Back class 는 Tagger 가 뒤돌아 볼 때의 상태를 의미하는데 이 Back class 는 TaggerState class 를 상속한다. front class 도 마찬가지 이다. 그리고 Player class 는 플레이어의 행동에 대한 관리와 게임 상태 (Win/Lose/keep)를 관리한다.

```
Red Light!  
  
Forward.  
start | @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@ | end ( 20/20 )  
  
Win!!
```

▷WIN 의 경우

20칸을 모두 전진하고 WIN을 했을 때의 화면이다.

```
Red Light!  
  
Tagger found out  
start | @@@@@@----- | end ( 3/20 )  
  
Lose..
```

▷LOSE 의 경우

그대로 게임이 종료되고 제대로 작동하는 것을 볼 수 있다.

이 문제는 그 동안 열심히 배웠던 class가 제일 많이 쓰인 문제라고 할 수 있을 것이다. Class 말고도 여러가지 동기성과 스레드 및 윈도우 헤더파일 그리고 키보드의 입력을 받아 플레이어의 동작을 제어하는 로직인 비동기 입력관리 타이머 시간관리, 상태 전이 등 여러가지 개념들을 새롭게 공부를 하게 한 문제였다. 처음 문제를 받았을 때는 게임 화면을 구성하라는 문제에 너무 당황스러웠지만, 올려주신 예제 코드와 헤더파일에 대해 공부하고 게임을 구현하기 위한 class 상태관리, 키보드 입력, 화면 출력 등을 공부하고 공개된 예제 코드의 함수들에 대해 찾아보고 공부하니 조금씩 프로그램을 구현할 수 있었고, 의외로 재미있게 과제를 수행할 수 있었다. 먼저 TaggerState class (상태 추상)에 대해서 설명하자면 순수 가상 함수를 정의를 사용하는데, 이는 상태의 패턴을 이용하여 술래의 상태를 관리 할 수 있다. 예를들어 Back class 는 Tagger 가 뒤돌아 볼 때의 상태를 의미하는데 이 Back class 는 TaggerState class 를 상속한다. front class 도 마찬가지로 이다. 그리고 Player class 는 플레이어의 행동에 대한 관리와 게임 상태 (Win/Lose/keep)를 관리한다.

문제 7번

1

문제설명

크게는 Stack class 를 구현하는 문제이다. Stack은 Linked List 로 구현하도록 하고, template 를 활용하여 다양한 유형의 데이터를 저장할 수 있도록 구현하는 문제이다. 구현해야할 기능들은 다음과 같다. < Push , pop, top, isEmpty, print >

2

알고리즘

● Node class

Linked List에서 각각의 노드에 대한 class 로 안에는 data와 next 포인터가 존재한다. 여기서 data는 타입형이 T 형이어야 한다.

● Stack class

private으로 스택이 최상위 노드를 가리키는 m_Top 포인터를 가진다. 그리고 stack 안에는 push 함수, pop 함수, is Empty 함수, top 함수, print 함수가 존재한다.

● Push

이 함수는 stack 에 새로운 인자를 삽입해 주는 함수로써 Node를 동적으로 생성한 후 해당 노드를 Top 앞 쪽에 삽입한다. Node->next 가 top 이 되도록 설정고 top을 node로 선언 해 준다. 스택은 LIFO 원칙을 따른다.

● isEmpty

이 함수는 Stack이 비었는지 안 비었는지 확인 해 주는 함수이다. 나중에 Underflow 를 확인해 줄 때도 필요한 함수이다. 그리고 사용자로부터 명령어를 입력받아도 empty여부를 확인해 값을 출력한다.

● Top

Top 함수는 사용자로부터 top command 를 입력 받았을 때 , 먼저 stack이 비었는지 여부를 isEmpty를 사용해 확인해 주고 Stack 이 빈게 아니라면 현재 최상위 Top Node의 data 값을 return 하여 준다. `return m_Top->data ;`

● print

print 함수는 사용자로부터 print 를 입력받았을 때 현재까지의 stack을 print 하는 함수이다. Print 하는 방식은 linked list 의 print 방식과 동일하다. 먼저 임시로 current 노드 포인터를 생성하고 top 노드를 가리키도록 한다. 그리고 current 가 null 이 아닐때까지 반복문을 돌도록 하고 그 때마다 current->data를 출력한다. 그리고 current는 current->next 로 이동시켜 주어 한 노드 씩 전진하며 모든 node를 출력하도록 한다.

● Stack 선언

Stack 선언은 다음과 같이 메인에 해준다.

```
Stack<int> s1;  
  
Stack<double> s2;  
  
Stack<char> s3;
```

왼쪽에 Stack< *** > 은 class 가 받을수 있는 자료형을 의미한다. 따라서 여러 input 타입에 따라 stack class 를 선언하고 stack을 쌓을 수 있다.

3

결과화면

Stack을 int 형으로 선언하였을 때

```
push 1  
push 2  
push 3  
push 4  
push 5  
print  
Stack : 5 -> 4 -> 3 -> 2 -> 1  
pop  
5
```

▷ push 가 순서대로 잘 되는 것을 볼 수 있다.

▷ pop을 하였을 때 가장 마지막으로 입력한 것이 출력되는것을 확인할 수 있다.

```
pop  
5  
print  
Stack : 4 -> 3 -> 2 -> 1
```

▷ pop 을 한 후 Stack에서 제대로 제거된 것을 확인할 수 있다.

```

push 1
push 2
push 3
push 4
push 5
pop
5print
Stack : 4 -> 3 -> 2 -> 1
top
4

```

▷ top = 4

pop으로 5를 제거 했기 때문에 현재 stack 은 4임을 확인 할 수 있다.

```

isEmpty
Stack is empty
push 1
push 2
push 3
isEmpty
Stack is not empty

```

▷ isEmpty 명령어 확인

Stack 비었는지 확인하는 명령어 이다. Stack 이 비었을 때는 Stack is empty 라고 출력되고, 아닌 경우에는 Stack is not empty 가 출력된다.

Stack을 **double 형**으로 선언하였을 때

```

push 3.3
push 4.5
push 1.7
push 9.9
push 11.8
print
Stack : 11.8 -> 9.9 -> 1.7 -> 4.5 -> 3.3
pop
11.8print
Stack : 9.9 -> 1.7 -> 4.5 -> 3.3
top
9.9

```

▷ double 형 input 들이 제대로 삽입되어 stack 에 쌓이는 것을 볼 수 있다. Pop을 했을 때 현재 최상위 인자가 무엇인지 확인하고 지워지는 것을 확인할 수 있다. Top은 현재 최상위 인자를 알려주는 명령어로 9.9가 제대로 출력되는 것을 확인 할 수 있다.

Stack을 **char** 형으로 선언하였을 때

```
push a
push q
push w
push e
push r
print
Stack : r -> e -> w -> q -> a
pop
rprint
Stack : e -> w -> q -> a
top
e
```

▷ char 형의 인자들이 push 를 통해 제대로 stack에 입력되고, pop 을 통해 최상위 인자가 지워지는 것을 확인 할 수 있다. 그리고 top 명령어를 실행하면 최상위 인자가 무엇인지 확인만 해주는 것을 확인 할 수 있다.

4

고찰

이번 문제를 풀며 template에 대해 처음으로 공부하고 알게 되었다. C++ 등에서 사용되는 템플릿을 활용해 여러 데이터 타입에 대해 stack을 사용할 수 있다는 것을 알게 되었다. 그리고 이번에 따로 입력에 대한 제약사항은 없었지만 사용자에게 하나씩 입력받아 stack을 쌓는 것으로 코드를 작성하였다. 그렇게 사용자로부터 실시간으로 입력받고 그에 따른 결과 출력하는 것이 확실히 가시적이고 재밌다고 생각해 그렇게 구현하였다. 저번 과제에서도 stack에 대해 공부하였기에 이번에는 보다 수월하게 과제를 수행할 수 있었다.

문제 8번

1

문제설명

이번 문제는 queue를 linked list 와 class 를 사용하여 구현하는 것이다. 큐의 특징은 먼저 들어온 데이터를 먼저 내보내는 선입선출 FIFO 구조를 가졌다. 이 문제에서 구현해야 하는 기능들은 다음과 같다. Enqueue, dequeue, front, isEmpty, print, exit 이다. Enqueue 는 stack 에서 push 와 같은 기능을 하고, dequeue 는 큐에서 가장 먼저 추가된 요소를 제거하고 반환하는 함수이다. 그리고 front 는 큐의 맨 앞의 인자를 꺼내서 보여준다. 지우거나 하지는 않는다. 그리고 isEmpty 는 queue 가 비었는지 여부를 알려주는 함수이고, print 는 현재까지의 queue를 출력하는 함수이다. 그리고 사용자로부터 exit 명령어를 입력받으면 프로그램을 종료한다.

2

알고리즘

● Enqueue

사용자로부터 enqueue 명령어를 입력 받는다면, 명령어와 함께 입력받은 데이터를 queue 에 삽입해 준다. Enqueue 명령어는 stack 에서는 push 명령어와 같은 기능을 수행하는 함수이다. 데이터를 queue 에 삽입하는 핵심 로직은 다음과 같다. Queue 는 맨 마지막 노드를 나타내는 back 노드가 존재한다. 이 back node 를 새로운 node로 설정 해주고, 마지막 노드를 node로 선언한다. 이 경우는 queue에 노드가 하나라도 존재할 때 이야기 이다. 만약 queue 가 비어 있다면, 입력받은 node 가 front node와 back node 모두 새로 입력 받은 node가 되도록 설정해 준다.

● dequeue

이 함수의 경우에는 stack에서는 pop 과 같은 함수이다. queue list에서 맨 위 data 를 제거 해 준후, 반환해 주는 함수이다. 핵심 로직은 다음과 같다. 큐 같은 경우에 dequeue 를 실행하면 제일 최근에 넣은 데이터를 지워야 한다. 따라서 먼저 temp 노드 포인터를 생성한 후 맨 위인 front를 가리키도록 한다. 그리고 제거할 노드의 데이터를 저장한다. 이 과정은 나중에 삭제한 데이터를 출력해 보이기 위해 필요한 과정이다. 그 후, front 가 temp->next 가 되도록 한다. 이렇게 하면 front 가 두 번째 노드를 가리키게 되고 첫 번째 노드는 삭제 된다.

그리고 만약 삭제 후에 queue 가 비었다면 back 도 null로 초기화 해 준다. 마지막으

로 잠시 생성했던 temp 노드를 메모리 해제 해준다. 마지막으로 data를 반환해준다.

● front

queue의 맨앞 노드 포인터인 front 를 반환 하는 함수이다. 이 함수는 front 를 보이
기만 하고, front를 삭제 하는 함수는 아니다. 따라서 queue가 비었는지 여부를 확인
한 후, 비지 않았다면 `return m_Front->getData();` 를 시행해주면 된다.

● isEmpty

이 함수 역시 stack에서와 같이 queue 가 비었는지 확인 해 주는 함수이다. 나중에
dequeue를 하거나 , newNode 를 삽입할 때 비었는지 안비었는지 여부에 따라 다르
게 해줘야 함으로 꼭 필요한 함수이다.

● Print

Print 함수의 출력 역시 Linked List 와 같다, 먼저 current 노드 포인터를 생성해 맨
앞 노드인 front 를 가리키도록 한다. 그리고 current 가 null 이 아닐 때까지 반복문
이 돌아가도록 조건을 취해 준 후, 반복문 안에는 current 의 data를 출력하고, 화살
표를 출력하고, current를 `current->next` 로 선언해 다음노드로 이동하도록 한다.

3

결과화면

```
enqueue 1
enqueue 2
enqueue 3
print
Queue: 1 -> 2 -> 3
```

4

고찰

이 문제는 7번 문제와 유사하게 class 와 template 을 사용하여 구현하는 문제이다. 7번 에서
는 stack을 구현했는데 이는 선입후출 구조이지만, 이번 queue 는 선입선출 FIFO 구조를 갖
추고 있다. 따라서 List 에 삽입할 때 이 부분만 고려하여 구현해 주면 7번과 거의 유사하게
구현이 가능하다. 그리고 queue 에는 front 와 back 모두 존재한다. 이 부분에서 원형 linked
list 문제가 생각이 났다. Linked list 와 전반적인 흐름은 같지만 원형 linked list 는 맨뒤를 가

리키는 tail(last) 노드 포인터가 존재하기 때문에 유사한 느낌이 들었다. 이렇게 스택과 큐에 관한 문제를 또 풀게 되자 이젠 스택 개념은 상당히 많이 이해한 것 같은 기분이 든다.

문제 9번

1

문제설명

BST (binary search tree)를 구현하는 문제이다. 문제를 구현하기 위해 BST class 와 각각의 Node 들을 관리하는 Node class 를 만든다. BST 란 입력받은 정수들을 정렬된 트리 형태로 출력하는 것이다. 문제에서 주어진 제약조건은 다음과 같다. 먼저 mid 노드를 만든 후 이를 root node로 둔다. Middle element 는 $(start + end) / 2$ 인덱스로 정의 한다. 그리고 mid 노드를 기점으로 왼쪽의 노드는 mid 보다 작은 mid 의 자식 노드가 되고, 오른쪽 노드는 mid 보다 큰 mid 의 자식 노드가 된다. 이 과정을 재귀적으로 구현하는 문제이다.

출력할 때의 제약조건은 다음과 같다. 출력시킨 줄에는 반드시 한 노드만이 존재할 수 있다. 그리고 만약 왼쪽 노드와 오른쪽 노드 모두 존재하지 않을 때에는 공백 두칸이 출력되어야 한다. 그리고 왼쪽과 오른쪽 노드는 mid를 기점으로 반드시 같은 거리만큼 떨어져 있어야 한다.

2

알고리즘

- Node class

Node class 에서는 member로 각 노드의 data 값과, 깊이 depth, 왼쪽 자식 노드 포인터인 left 와 오른쪽 자식 노드 포인터인 right 를 가진다. 생성자는 노드를 생성할 때 데이터와 깊이를 초기화 하도록 한다. 자식 노드는 nullptr 로 초기화 한다. 소멸자는 노드를 파괴할 때 재귀적으로 파괴한다.

- BST class

BST class 에는 member 변수로 트리의 첫 노드인 루트를 가리키는 m_root 와 트리의 최대 깊이를 의미하는 maxDepth 를 가진다.

- Build 함수

입력 받은 배열을 기반으로 binary tree를 구축한다. 이 함수의 핵심 로직은 배열의 mid 를 찾아 트리에 삽입하고, mid 를 기준으로 왼쪽과 오른쪽 자식 노드에 대해 재

귀적으로 반복한다.

- Insert 함수

Insert 함수는 주어진 depth 깊이와 node 값을 사용해 새로운 노드를 적절한 위치에 삽입한다. Insert 함수의 핵심 로직은 다음과 같다. 먼저 root 가 없다면 새로운 노드를 루트 노드로 설정해 준다. 그렇지 않다면 , insert 하기 올바른 위치를 찾아 탐색한다. 값을 비교하여 값이 노드의 값보다 작으면 왼쪽 자식노드로 배치하고, 값이 보다 더 크다면 오른쪽 자식 노드로 이동 시킨다. 그리고 각 노드와 max 깊이를 재설정 한다.

- printTree 함수

트리를 출력하는 함수이다. printRec를 이용하여 재귀적으로 호출한다. 먼저 루트노드에서 시작해 오른쪽 자식 노드부터 재귀적으로 호출한다. 왜냐하면 문제에서 요구한것이 오른쪽 자식 노드가 위쪽에 출력되어야 하기 때문이다. 같은 방식으로 왼쪽 자식 노드도 출력한다.

3

결과화면

```
Enter the size of the input array: 10
Enter 10 elements of the input array (sorted in ascending order):
12 21 37 46 59 65 74 83 89 94
      94
    89
  83
    74
  65
59    46
    37
  21
    12
```

▷ 문제 예제와 같은 숫자를 입력하였을 때 올바르게 결과가 나오는 것을 확인 할 수 있다.

```
Enter the size of the input array: 10
Enter 10 elements of the input array (sorted in ascending order):
12 21 37 46 59 65 74 83 89 94
insert: 59
루트 생성

insert: 21
left 이동

insert: 12
left 이동
left 이동

insert: 37
left 이동
right 이동

insert: 46
left 이동
right 이동
right 이동

insert: 83
right 이동

insert: 65
right 이동
left 이동

insert: 74
right 이동
left 이동
right 이동

insert: 89
right 이동
right 이동

insert: 94
right 이동
right 이동
```

▷ 이것은 노드의 이동이 어떻게 되는지 궁금해서 찍어본 결과 화면이다. 올바르게 노드가 배치되는 모습이다.

```

Enter the size of the input array: 7
Enter 7 elements of the input array (sorted in ascending order):
10 20 30 40 50 60 70
      70
    60
  50
40
    30
  20
    10

```

▷ 배열의 크기가 7일 때, 역시 제대로 출력되는 것을 확인 할 수 있다.

```

Enter the size of the input array: 5
Enter 5 elements of the input array (sorted in ascending order):
55.5 67.6 68.9 77.7 89.9
      89.9
    77.7
  68.9
    67.6
  55.5

```

▷ 실수를 넣어도 제대로 출력되는 것을 확인할 수 있다.

4 고찰

이전에도 Binary search는 과제로 수행한 적이 있었다. 그 때에는 정렬된 함수에서 값을 찾기 위해 이진법으로 값을 찾는 것이었다. 이번에도 핵심 로직은 비슷하다 할 수 있다. Mid를 기점으로 한다는 점에선 말이다. 그러나 그 때는 왼쪽 이나 오른쪽이나 필요없는 노드는 버리고 해당 노드를 다시 mid를 기점으로 나누었다. 이번에는 모든 노드를 버리지 않고 모두 mid를 기점으로 노드를 나누고, 부모 - 자식 관계를 주어 트리를 형성하도록 하였다. 이렇게 가시적으로 내가 코딩한 것들이 보이자 너무 흥미로웠다. 이 과제를 수행하기 위해 BST에 대해서도 찾아보고 공부하였는데, 여러 출력모양의 트리들이 있었고, 이러한 로직을 조금 변형하여 여러 개의 관계가 오가는 경우에는 그래프 라고 하는 개념도 알게 되었다. 이러한 것들을 공부하고 활용하면서 내 능력치가 보다 상승한 것 같아 매우 뿌듯했다.